Videoconferência Volumétrica Baseada em WebRTC: Avaliação e Benchmarking de Arquiteturas SFU

Matthias De Fré[1] · Jeroen van der Hooft[1] · Jack Jansen[2] · Silvia Rossi[2] · Thomas Röggla[2] · Tim Wauters[1] · Filip De Turck[1] · Irene Viola[2] · Pablo Cesar[2,3]
[1]IDLab, Ghent University – imec, Gante, Bélgica · [2]Centrum Wiskunde & Informatica, Amsterdã, Países Baixos · [3]TU Delft, Delft, Países Baixos

NOSSDAV '26 (36ª edição do Workshop on Network and Operating System Support for Digital Audio and Video), 4–8 de abril de 2026, Hong Kong. DOI: 10.1145/3798065.3798070. Licença original: CC BY 4.0.

Tradução não oficial para o português (pt-BR) de *WebRTC-Based Volumetric Video Conferencing: SFU Architecture Evaluation and Benchmarking*. Termos técnicos consagrados (como *bitrate, throughput, jitter*) foram mantidos em inglês quando soam mais naturais assim. Figuras e imagens foram omitidas (permanecem apenas as legendas traduzidas); tabelas e equações foram reproduzidas. A lista de referências é mantida no idioma original. Em caso de dúvida, vale o texto original.

RESUMO

As tecnologias imersivas prometem revolucionar a comunicação por meio de uma maior sensação de presença e de interatividade. Para viabilizar a interação, são necessários mecanismos de transporte confiáveis e de baixa latência para lidar com os grandes volumes de dados criados por objetos 3D complexos. Neste artigo, propomos uma unidade de encaminhamento seletivo (SFU) open source e independente de codec para streaming de vídeo volumétrico em tempo real usando WebRTC. Para fins de avaliação, fornecemos uma implementação de cliente de referência, estendendo o VR2Gather, um sistema baseado em TCP para comunicação imersiva. Realizamos avaliações extensas, com datasets novos e existentes, para comparar o desempenho do WebRTC com o de protocolos baseados em TCP em um ambiente de testbed emulado. As avaliações demonstram que o WebRTC supera os demais protocolos em cenários de alta latência e adapta a qualidade do vídeo ao movimento do usuário 13% e 36% mais rápido que seus equivalentes baseados em TCP, em redes com 5 ms e 10 ms de latência de rede, respectivamente.

Palavras-chave: Vídeo volumétrico, conferência, realidade virtual, WebRTC.
Conceitos CCS: Sistemas de informação → Streaming multimídia

Figura 1: A SFU WebRTC proposta, independente de codec, permite o encaminhamento seletivo de níveis de qualidade por cliente.
(imagem omitida nesta tradução; consulte o artigo original)

1 INTRODUÇÃO

As tecnologias imersivas estão redefinindo a comunicação remota ao aumentar a copresença e a interatividade, tornando as interações virtuais cada vez mais parecidas com as presenciais [16]. Os protocolos de transmissão desempenham um papel fundamental na viabilização da comunicação volumétrica em tempo real, permitindo escolher entre a confiabilidade, ao usar TCP, ou a menor latência, com UDP [2]. O WebRTC, baseado em UDP, é comumente usado para oferecer streaming de baixa latência, com mecanismos opcionais de confiabilidade [8]. Embora o modelo ponto a ponto (*peer-to-peer*) do WebRTC viabilize o streaming, surgem problemas de escalabilidade à medida que o número de usuários cresce, o que motiva soluções centralizadas como as unidades de encaminhamento seletivo (SFUs). Apesar do recente foco em vídeo volumétrico, até onde sabemos, nenhum trabalho anterior comparou diretamente o WebRTC com soluções baseadas em TCP dentro do mesmo software de referência, já que os dois raramente são implementados juntos [25].

Para suprir essas lacunas, propomos uma SFU baseada em WebRTC, independente de codec e open source [27], para streaming volumétrico muitos-para-muitos em tempo real. Como mostra a Figura 1, os clientes capturam, segmentam e codificam os dados volumétricos em múltiplas qualidades, que a SFU encaminha seletivamente com base nas condições da rede e do usuário. Os receptores então decodificam e renderizam as *tracks* em tempo real, apoiados por mensagens de feedback para adaptação de qualidade e sincronização. A solução proposta pode ser integrada a plataformas de streaming volumétrico. Para fins de avaliação, é fornecida uma implementação de cliente de referência baseada no VR2Gather [13], que serve de referência para o benchmarking de protocolos baseados em UDP contra os baseados em TCP (TCP direto, SocketIO e LL-DASH) na comunicação imersiva. Em resumo, as principais contribuições deste artigo são:

- Uma SFU baseada em WebRTC e independente de codec para conferência volumétrica com qualidade adaptativa;
- Uma avaliação do sistema com múltiplos protocolos de rede, esquemas de qualidade e latências de rede;
- Uma implementação de cliente WebRTC de referência, mostrando como a solução pode ser implantada em diferentes plataformas;
- Um dataset bidirecional realista de dois usuários.

2 TRABALHOS RELACIONADOS

Esta seção apresenta sistemas de conferência volumétrica, revisa trabalhos relacionados sobre codecs de nuvem de pontos e discute o streaming adaptativo com os codecs existentes.

Sistemas de videoconferência volumétrica. Muitas plataformas imersivas foram projetadas para facilitar a conferência volumétrica. As soluções de alta fidelidade oferecem qualidade ótima ao usuário, mas exigem configurações complexas [15]. Em contraste, sistemas mais escaláveis usam qualidade reduzida, com escalabilidade baseada em unidades de controle multiponto (MCU) [11], para melhorar o desempenho do sistema. Em [5], o desempenho é melhorado ainda mais com o uso de adaptação de qualidade baseada nas condições da rede e no campo de visão (FoV) para reduzir a latência. Por fim, o VR2Gather [13, 25] oferece uma plataforma modular para comunicação volumé-

trica, com suporte a vários protocolos de transporte confiáveis, mas sem soluções baseadas em UDP. Por isso, adotamos o VR2Gather para avaliar nossa implementação WebRTC, o que permite fazer o benchmarking da transmissão baseada em UDP contra a baseada em TCP.

Fundamentos do WebRTC. O WebRTC é comumente usado por sua baixa latência e por sua escalabilidade potencial com arquiteturas baseadas em servidor. Sistemas baseados em MCU mesclam todos os fluxos de entrada em um único fluxo por cliente, mas essa agregação é computacionalmente cara e intensiva em memória, limitando a escalabilidade e aumentando a latência pela necessidade de esperar por todas as entradas [10]. Em contraste, arquiteturas baseadas em SFU encaminham os pacotes direta e imediatamente aos clientes relevantes, reduzindo a latência e o uso de memória e permitindo melhor *pacing* de pacotes, o que melhora a qualidade geral do fluxo de rede e diminui a probabilidade de perda de pacotes. O WebRTC estabelece conexões entre pares por meio da troca de mensagens do protocolo de descrição de sessão (SDP), que descrevem os parâmetros da sessão. Usando um modelo oferta–resposta, um par envia uma oferta SDP com as configurações suportadas, enquanto o outro responde com uma resposta SDP que seleciona os parâmetros compatíveis [20].

Compressão de nuvens de pontos. As altas taxas de bits das nuvens de pontos não comprimidas exigem altos níveis de compressão [23]. Codecs fora do tempo real, como o de compressão de nuvens de pontos baseada em vídeo (V-PCC), alcançam taxas de bits baixas, semelhantes às do vídeo 2D, mas a um alto custo computacional [9]. Em contraste, o Draco [7] foca na codificação em tempo real, ao custo de taxas de bits várias vezes maiores que as do V-PCC. O codec cwipc estende o codec âncora do MPEG [17], armazenando a geometria em uma octree e codificando a cor via compressão JPEG. O codec oferece um bom equilíbrio entre eficiência e velocidade, alcançando desempenho em tempo real a 15 FPS com nuvens de pontos de 200.000 pontos, o que o torna bem adequado ao streaming volumétrico ao vivo.

Streaming adaptativo de vídeo imersivo. Como o vídeo volumétrico impõe grandes demandas de largura de banda e de computação, várias soluções têm se concentrado em otimizar a transmissão para os usuários finais [26]. O streaming adaptativo tradicional foi estendido para levar em conta a interatividade, selecionando representações com base na direção de visão e nas condições da rede [19, 28]. Outra estratégia segmenta o conteúdo em tiles codificados em múltiplas qualidades e entregues via DASH, permitindo que os clientes solicitem os níveis de qualidade apropriados [21, 22, 24]. Codecs como o cwipc suportam nativamente tiling multiqualidade, enquanto outras abordagens usam métodos baseados em codificação por múltiplas descrições (MDC), independentes de codec, para combinar camadas de qualidade [5]. Neste artigo, fornecemos uma solução independente de codec para entregar vídeo volumétrico usando WebRTC, a fim de alcançar menor atraso de ponta a ponta e oferecer um meio de adaptação de qualidade com base nas condições da rede.

3 PROJETO ARQUITETURAL E IMPLEMENTAÇÃO

Nesta seção, descrevemos os conceitos centrais do conjunto de protocolos WebRTC, apresentamos as decisões de projeto de alto nível da nossa SFU independente de codec e descrevemos sua implementação.

Arquitetura. A conferência volumétrica muitos-para-muitos de alto desempenho exige uma arquitetura mediada. Para evitar o custo computacional e a latência dos projetos baseados em MCU, o sistema proposto adota uma arquitetura SFU. Como mencionado na Seção 2, o streaming adaptativo reduz a carga da rede e melhora a qualidade. O suporte nativo do WebRTC a múltiplas *tracks* permite que a SFU transmita dados por fluxos distintos, possibilitando que as nuvens de pontos sejam particionadas por tiling ou segmentação semântica e entregues como *tracks* separadas. Além disso, a codificação paralela pode ser usada para gerar várias qualidades, cada uma transmitida por uma *track* separada. Como essas *tracks* representam o mesmo conteúdo, a SFU encaminha no máximo uma por cliente, selecionando a qualidade apropriada com base na largura de banda e no contexto do usuário. Embora cada objeto tenha uma *track* de transmissão para cada qualidade, cada cliente receptor mantém apenas uma única *track* por objeto. Esse mecanismo minimiza as *tracks* abertas e a carga computacional, ao mesmo tempo que permite à SFU trocar de qualidade reatribuindo o remetente na *track* de encaminhamento existente, sem renegociação SDP. Em conjunto, esses mecanismos suportam diversas estratégias de codificação, incluindo esquemas de segmentação com múltiplas qualidades por segmento e abordagens MDC que combinam segmentos para melhorar a qualidade.

Figura 2: Setup físico no testbed Virtual Wall [12] (um servidor, dois switches e dois clientes).
(imagem omitida nesta tradução; consulte o artigo original)

Implementação da SFU. A SFU é implementada adaptando o Pion, em Golang [18], que fornece o conjunto completo de protocolos WebRTC e a estimativa de largura de banda no lado do servidor via Google Congestion Control (GCC). Uma *track* de mídia personalizada cuida da transcodificação e da empacotamento dos quadros de nuvem de pontos por meio de um *payloader* dedicado, enquanto o áudio é enviado por uma *track* separada. São usadas transmissões baseadas em reconhecimento negativo (NACK) para garantir que os quadros sejam decodificáveis em caso de perda de pacotes. Combinada ao GCC, essa abordagem demonstrou bom desempenho no streaming de vídeo volumétrico em redes congestionadas [4]. Ao se conectar, o servidor sinaliza as *tracks* do novo cliente aos demais pares, levando-os a instanciar as *tracks* necessárias para a transmissão. O servidor também suporta seleção de qualidade e permite o suporte a mensagens de controle adicionais da SFU com sobrecarga mínima. Para permitir a troca de qualidade sem interrupções, um interceptador de pacotes personalizado preserva a numeração de sequência entre as *tracks* de qualidade. Sem esse interceptador, lacunas na sequência seriam interpretadas como perda de pacotes, disparando retransmissões desnecessárias da *track* de qualidade errada. O uso de um servidor opcional de utilitários de travessia de sessão para NAT (STUN) mitiga problemas em redes que empregam tradução de endereços de rede (NAT) [6].

Implementação de referência do peer WebRTC. Qualquer biblioteca cliente WebRTC que suporte *tracks* de mídia com codecs não padronizados pode ser usada para se conectar à SFU proposta. No entanto, para aproveitar plenamente os recursos da SFU, o cliente também precisa suportar NACK, GCC e as mensagens de feedback associadas. Após uma avaliação abran-

gente das bibliotecas WebRTC disponíveis, com os critérios mencionados acima, a biblioteca Pion foi selecionada para a implementação de cliente de referência. Contudo, a linguagem Golang não é nativamente compatível com o Unity, o motor de jogos empregado pelo VR2Gather. Para contornar essa lacuna, foi desenvolvida uma biblioteca que grava os quadros de saída em uma porta UDP local, consultada continuamente pelo cliente em Golang. A sobrecarga adicional introduzida por essa abordagem foi medida em aproximadamente 1 ms, um custo desprezível diante dos benefícios de desempenho que ela viabiliza.

4 METODOLOGIA DE AVALIAÇÃO E CONFIGURAÇÃO

Nesta seção, primeiro discutimos a metodologia de avaliação, descrevendo os experimentos realizados. Em seguida, são apresentados a configuração experimental e as métricas consideradas.

4.1 Metodologia de avaliação

O VR2Gather atualmente suporta quatro protocolos: TCP direto (*Direct TCP*), TCP Reflector, SocketIO e a implementação WebRTC proposta. Cada protocolo permite habilitar os seguintes módulos: compressão, codificação multiqualidade e tiling.

Em cenários unidirecionais, consideram-se um único remetente, responsável pela reprodução e pela codificação, e um único receptor, responsável pela decodificação e pela renderização. O objetivo é avaliar a eficiência dos protocolos na plataforma VR2Gather, ao mesmo tempo em que se avalia o impacto da taxa de quadros de codificação, do nível de qualidade e do tiling sobre o desempenho do sistema.

Em cenários interativos, os experimentos comparam a latência de adaptação de qualidade ao usar WebRTC ou TCP direto, os únicos protocolos do VR2Gather que suportam adaptação de qualidade com desempenho aceitável. Nessa configuração bidirecional, ambos os clientes reproduzem conteúdo gravado e usam movimentos capturados para conduzir as decisões de adaptação de qualidade [1]. Cada cliente opera com uma taxa de quadros diferente, para avaliar o impacto da taxa de quadros de envio sobre a latência de adaptação de qualidade do outro cliente. Para permitir uma comparação justa com trabalhos anteriores baseados no VR2Gather [14], é usada a lógica de decisão já existente do VR2Gather e as decisões de qualidade são trocadas via websocket, em vez de depender de um estimador de largura de banda dedicado.

4.2 Configuração experimental

A Figura 2 mostra a topologia considerada, na qual dois clientes são conectados a um servidor centralizado por meio de switches personalizados. As máquinas envolvidas são interconectadas por um enlace de 1 Gb/s, e cada uma tem uma das seguintes especificações:

- CPU: Intel Xeon E3-1220 v3, RAM: 16 GB DDR3
- CPU: Intel Xeon E3-5645 v3, RAM: 24 GB DDR3

Como esses nós não têm os recursos necessários para renderização, laptops de maior desempenho são conectados diretamente aos nós cliente por VPN. Essa configuração permite forçar os laptops a seguirem uma rota específica pela rede, o que garante que quaisquer degradações de rede aplicadas ao tráfego que passa pela rede também sejam aplicadas durante os experimentos. Todas as máquinas estão na mesma rede e, portanto, introduzem latência inferior a 1 ms, além de serem sincronizadas pelo protocolo de tempo de rede (NTP).

4.3 Métricas de avaliação

Para avaliar com precisão o desempenho do sistema, cada experimento é realizado com 10 iterações, cada uma com duração de cerca de 1 minuto. Consideramos as seguintes métricas (com intervalos de confiança de 95%): latência do pipeline, largura de banda e uso de CPU, FPS no receptor e latência de mudança de qualidade (tempo entre a solicitação de uma mudança de qualidade e a renderização dessa qualidade).

O seguinte hardware é usado:

- Cliente 1: CPU: Intel Core i7-12700H (2,30 GHz), RAM: 32 GB DDR5, GPU: NVIDIA GeForce RTX 3070 Ti
- Cliente 2: CPU: Intel Core i7-10750H (2,60 GHz), RAM: 16 GB DDR4, GPU: NVIDIA GeForce RTX 2070

Para emular condições de redes de longa distância, é aplicada nos nós comutadores uma latência bidirecional entre 0 e 50 ms, correspondente à metade do tempo de ida e volta (RTT) cliente–servidor.

Para os experimentos unidirecionais, considera-se a sequência Loot do dataset 8i [3], a 15 e 30 FPS. As nuvens de pontos são pré-escaladas com um tamanho de voxel de 2,5 para atingir uma contagem de 200.000 pontos, igual à de [25], e são divididas em quatro tiles conforme descrito em [21]. O codec âncora do MPEG [17] é usado para permitir a codificação em tempo real, com dois níveis de qualidade (baixa qualidade = octree 7 e alta qualidade = octree 9). Nesses experimentos, o cliente 1 é o remetente e o cliente 2 é o receptor.

Para os experimentos interativos, o conteúdo foi capturado com duas máquinas e quatro câmeras Microsoft Azure Kinect, resultando em quinze tiles. Para provocar mudanças de qualidade suficientes, dois usuários dançam juntos e giram um em torno do outro durante uma sessão de um minuto. Para demonstrar a flexibilidade da plataforma, um usuário é capturado a 15 FPS e o outro a 30 FPS, com ambos os clientes atuando como remetente e receptor. Como a codificação e a decodificação ocorrem em cada máquina, são usados níveis de qualidade reduzidos (baixa qualidade = octree 5 e alta qualidade = octree 7).

5 RESULTADOS E DISCUSSÃO

Nesta seção, os resultados são discutidos. Primeiro, os resultados do cenário unidirecional, junto com um exame do impacto do tiling e da latência de rede. Segundo, os resultados do cenário bidirecional, com foco nas capacidades de tempo real da troca de qualidade.

5.1 Benchmark de conteúdo unidirecional

Impacto dos protocolos e da qualidade. A Tabela 1 apresenta os resultados do experimento com conteúdo sem tiling a 15 FPS para cada protocolo. Todos os protocolos apresentam valores de latência dentro de uma margem de 5 ms. Além disso, durante cada iteração a latência é estável, com pequenas flutuações atribuíveis a componentes que não são de transporte. O WebRTC apresenta largura de banda ligeiramente maior. Esse aumento decorre de quadros maiores, que levam a mais pacotes, cada um com a sobrecarga adicional de cabeçalhos do WebRTC. Além disso, a Tabela 1 revela que o streaming em qualidade mais alta (octree 9) impede o sistema de atingir a taxa de quadros alvo de 15 FPS com o hardware utilizado.

Tabela 1: Desempenho do sistema em termos de latência, largura de banda, taxa de quadros e uso de CPU em sessões com dois participantes que trocam o objeto Loot sem latência de rede adicional.

Protocolo	Tiles	Nível de octree	FPS alvo	Latência (ms)	Banda (Mb/s)	FPS obtido	CPU
TCP direto	1	7	15	45,29 ± 4,15	3,70 ± 0,37	14,90 ± 0,42	39,88 ± 8,27
TCP direto	1	9	15	334,20 ± 25,66	11,17 ± 1,68	7,18 ± 0,86	44,32 ± 19,63
TCP Reflector	1	7	15	46,11 ± 2,73	3,75 ± 0,34	14,89 ± 0,40	39,95 ± 8,05
TCP Reflector	1	9	15	332,01 ± 26,88	11,36 ± 1,64	7,24 ± 0,86	43,64 ± 26,25
SocketIO	1	7	15	50,46 ± 3,48	3,69 ± 0,37	14,98 ± 0,64	42,47 ± 17,67
SocketIO	1	9	15	337,40 ± 32,90	11,33 ± 1,45	7,59 ± 0,78	45,10 ± 26,14
WebRTC	1	7	15	43,75 ± 3,19	3,94 ± 0,38	14,89 ± 0,46	40,69 ± 8,21
WebRTC	1	9	15	334,07 ± 27,54	12,28 ± 1,63	7,51 ± 0,83	45,89 ± 23,86

Tabela 2: Desempenho do sistema em termos de latência, largura de banda, taxa de quadros e uso de CPU em sessões com dois participantes que trocam o objeto Loot via WebRTC, para diferentes configurações de qualidade de vídeo.

Tiles	Nível de octree	FPS alvo	Latência (ms)	Banda (Mb/s)	FPS obtido	CPU
1	7	15	43,75 ± 3,19	3,94 ± 0,38	14,89 ± 0,46	40,69 ± 8,21
4	7	15	36,02 ± 1,86	8,09 ± 0,67	14,90 ± 0,45	42,86 ± 8,16
1	9	15	334,07 ± 27,54	12,28 ± 1,63	7,51 ± 0,83	45,89 ± 23,86
4	9	15	111,07 ± 9,42	25,73 ± 2,79	14,99 ± 0,99	49,99 ± 19,01
1	7	30	69,14 ± 1,58	7,63 ± 0,72	30,07 ± 0,85	51,69 ± 8,84
4	7	30	69,99 ± 1,74	15,71 ± 1,52	30,04 ± 0,83	52,69 ± 7,41
1	9	30	289,48 ± 16,42	12,05 ± 1,54	7,38 ± 0,60	38,29 ± 19,66
4	9	30	161,68 ± 14,81	31,47 ± 4,18	18,77 ± 2,41	66,51 ± 30,56

Impacto do tiling e da taxa de quadros. Como mostra a Tabela 2, o tiling tem impacto significativo sobre a latência do sistema, ao custo de aumentar a largura de banda total. Graças à codificação paralela de cada tile, os tempos de codificação e de decodificação são reduzidos por fatores de 4,63 e 3,47, respectivamente, o que torna possível alcançar taxas de quadros maiores e transmitir conteúdo de maior qualidade com taxas de quadros aceitáveis. No entanto, há uma exceção: ao usar uma profundidade de octree de 7 a 30 FPS, não se obtém mudança na latência. Embora o tempo de codificação diminua, o VR2Gather armazena quadros em buffer internamente para manter um FPS estável, o que resulta em nenhuma redução de latência. Como mostra a Figura 3, a maior redução de latência é observada para níveis de octree mais altos, o que mostra que o tiling elimina qualquer latência causada pela fila do codificador. Na prática, cada quadro é codificado e enviado imediatamente após ser produzido, mantendo a taxa alvo de 15 FPS. Essa melhoria permite que remetente e receptor sustentem 15 FPS com profundidade de octree 9. Entretanto, ao aumentar a taxa de quadros para 30 FPS, o remetente alcança apenas uma taxa de envio de 18 FPS, contra a taxa de captura de 30 FPS. Para conteúdo sem tiling, com profundidade de octree 9 e WebRTC, a Tabela 2 mostra que os valores de latência são menores com taxa de captura de 30 FPS do que com 15 FPS, apesar de ambos alcançarem a mesma taxa de quadros de renderização. Esse comportamento ocorre devido à capacidade limitada da fila do codificador. Quando um quadro é adicionado a uma fila cheia, o quadro mais antigo que não está sendo codificado no momento é descartado. Com taxas de captura maiores, os quadros entram na fila com mais frequência, levando a descartes mais frequentes. Consequentemente, os quadros que não são descartados passam 56% menos tempo no codificador com FPS alvo de 30 do que com 15 FPS. Isso demonstra que, em certos cenários, usar uma taxa de captura maior pode ser vantajoso mesmo que o cliente não consiga codificar quadros na mesma taxa.

Figura 3: Contribuições para a latência em sessões com dois participantes e taxa de quadros alvo de 15 FPS usando WebRTC. (a) Sem tiling, octree nível 7; (b) com tiling, octree nível 7; (c) sem tiling, octree nível 9; (d) com tiling, octree nível 9. Componentes: fila do transmissor, fila do renderizador, filas dos decodificadores, fila do codificador, latência máxima do renderizador, latência do renderizador, decodificadores e codificador.
(imagem omitida nesta tradução; consulte o artigo original)

Impacto da latência de rede. A Figura 4 mostra o impacto da latência de rede. Os resultados indicam que a maioria dos protocolos alcança resultados semelhantes, com uma exceção em latências altas: o TCP Reflector. Com 50 ms de latência adicionada, esse protocolo apresenta uma latência 21% maior que a do WebRTC. Além disso, para conteúdo com tiling a 15 FPS, a latência do TCP Reflector é maior que a dos demais protocolos sem tiling. Essa disparidade se torna menos pronunciada a 30 FPS, em que a latência do TCP Reflector permanece 10% maior que a do WebRTC, mas ainda supera o desempenho sem tiling dos demais protocolos. De forma semelhante aos resultados da Tabela 2, nenhum protocolo mostra redução de latência com o uso de tiling a 30 FPS sem atraso de rede induzido. Quando uma pequena quantidade de latência é introduzida, o atraso permanece inalterado por causa do buffering na fila de codificação do VR2Gather. No entanto, quando a latência induzida chega a 20 ms, o tiling leva a uma redução mensurável da latência total, com o TCP direto e o WebRTC superando os outros dois protocolos e alcançando, sem tiling, latência comparável à que os demais protocolos obtêm com tiling.

Figura 4: Contribuições para a latência para diferentes configurações de qualidade de vídeo e valores de latência de rede induzida, em sessões com dois participantes a 15 FPS e 30 FPS, usando profundidade de octree 7. (a) Conteúdo a 15 FPS; (b) conteúdo a 30 FPS. Curvas: TCP direto, TCP Reflector, SocketIO e WebRTC, com e sem tiling.
(imagem omitida nesta tradução; consulte o artigo original)

5.2 Latência de decisão de qualidade em sessões interativas bidirecionais

No VR2Gather, o tiling é usado para alcançar adaptação de qualidade atribuindo a maior qualidade ao tile mais próximo e uma qualidade menor aos demais tiles. Isso é crucial em cenários com muito movimento, como a dança destes experimentos. A Figura 5 mostra o impacto das mudanças de qualidade com latência de rede de 10 ms. Essas mudanças de qualidade não são imediatamente visíveis, o que pode reduzir a experiência geral [22]. Consequentemente, o tempo necessário para que uma mudança de qualidade se torne visível é um indicador valioso da experiência. Com o TCP direto, ocorrem picos frequentes de latência durante as mudanças de qualidade, comportamento não observado com o WebRTC. Essa diferença é atribuída aos mecanismos distintos de mudança de qualidade dos dois protocolos. Na implementação baseada em WebRTC, cada tile é associado a uma *track* WebRTC separada. No lado do receptor, é criada uma única *track* para cada tile, o que permite ao receptor simplesmente sinalizar à SFU a qualidade desejada, e a SFU então troca a *track* do remetente que está sendo encaminhada para a *track* do receptor. Essa abordagem minimiza a sobrecarga durante as mudanças de qualidade, reduzindo a probabilidade de picos de latência. Em contraste, a implementação de TCP direto no VR2Gather exige fechar e reabrir o socket de um tile sempre que ocorre uma mudança de qualidade, levando aos picos de latência observados. Esse processo envolve vários round-trips para transmitir as novas preferências de qualidade, resultando em picos de latência até que a conexão seja estabelecida. Entretanto, a vantagem da implementação de TCP direto é o uso eficiente da largura de banda, já que apenas as qualidades solicitadas pelo receptor são transmitidas. Em contraste, a implementação WebRTC sempre envia todos os níveis de qualidade à SFU, que então determina a qualidade apropriada a encaminhar aos receptores. Como o algoritmo de adaptação atual do VR2Gather seleciona apenas um tile para ter a maior qualidade, a largura de banda usada para transmitir os outros 14 tiles de alta qualidade é desperdiçada. Porém, como a solução WebRTC sempre encaminha todas as qualidades à SFU, o tempo de resposta para implementar a mudança de qualidade é significativamente menor. Como o WebRTC exige apenas meio RTT para sinalizar mudanças de qualidade, os ajustes frequentemente podem ser aplicados antes que o próximo quadro seja produzido. Esse comportamento é evidente na Figura 6, em que a latência de escolha permanece quase idêntica para atrasos de 5 ms e 10 ms ao usar WebRTC. Em contraste, o TCP direto exige múltiplos RTTs para implementar uma mudança de qualidade, o que aumenta significativamente a latência. Mesmo com um atraso de 5 ms, o TCP direto não consegue estabelecer uma nova conexão antes do próximo quadro. Além disso, ambos os protocolos têm desempenho significativamente melhor em comparação com os resultados do LL-DASH do VR2Gather, disponíveis publicamente [14]. No LL-DASH, a adaptação de qualidade ocorre por segmento, e não por quadro, o que torna a latência dependente do tamanho do segmento. Embora reduzir o tamanho possa mitigar esse problema, alcançar desempenho comparável ao do WebRTC exigiria segmentos extremamente pequenos, o que imporia uma sobrecarga substancial.

Figura 5: Contribuições para a latência para TCP direto e WebRTC com adaptação de qualidade, usando a sequência de dança capturada, com a primeira qualidade em profundidade de octree 7 e a segunda em profundidade 9, com 10 ms de latência adicionada. (a) Participante 1 a 15 FPS; (b) participante 2 a 30 FPS; (c) participante 1 a 15 FPS; (d) participante 2 a 30 FPS.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 6: Tempo entre a decisão de qualidade ser tomada e a nova qualidade escolhida ficar visível.
(imagem omitida nesta tradução; consulte o artigo original)

6 CONCLUSÃO E TRABALHOS FUTUROS

Neste artigo, propomos uma unidade de encaminhamento seletivo (SFU) open source baseada em WebRTC para conferência volumétrica muitos-para-muitos. Além disso, apresentamos uma implementação de cliente de referência baseada na plataforma VR2Gather, com suporte à adaptação de qualidade já existente no VR2Gather. Em redes sem latência, nossa abordagem alcança desempenho semelhante ou melhor em comparação com os protocolos baseados em TCP existentes no VR2Gather. Com 50 ms de latência induzida, a SFU proposta reduz a latência em 21% em comparação com o TCP Reflector, igualando os demais protocolos. Usando vídeos capturados e traços de movimento disponíveis publicamente, mostramos também que nossa SFU WebRTC combinada com a adaptação de qualidade do VR2Gather produz atualizações de qualidade visíveis mais rapidamente: melhorias de 13% e 36% com 5 ms e 10 ms de latência induzida, respectivamente. Além disso, pequenas variações de latência de 5 ms aumentam em 35% o tempo necessário para que uma troca de qualidade se torne visível no TCP direto, enquanto o WebRTC mantém desempenho estável.

Em trabalhos futuros, estenderemos o benchmark para um número maior de usuários, a fim de avaliar melhor a escalabilidade do sistema. O algoritmo atual de adaptação de qualidade do VR2Gather assume largura de banda fixa, o que limita a aplicabilidade a condições do mundo real. Como a plataforma já suporta estimativa de largura de banda, a integração de decisões de qualidade no lado do servidor melhoraria a adaptabilidade a condições de rede variáveis.

AGRADECIMENTOS

Este projeto recebeu financiamento do Horizon Europe Research and Innovation: projeto SPIRIT (Acordo de Subvenção 101070672, https://www.spirit-project.eu/), INDUX-R (Acordo de Subvenção Nº 101135556, https://indux-r.eu/) e TRANSMIXR (Acordo de Subvenção Nº 101070109, https://transmixr.eu/).

REFERÊNCIAS (NO IDIOMA ORIGINAL)

[1] [SW], Bidirectional Dataset Used in Experiments 2025. url: https://cloud.ilabt.imec.be/index.php/s/89HFrPET4gpX7CM.
[2] C. Cortés, I. Viola, J. Gutiérrez, J. Jansen, S. Subramanyam, E. Alexiou, P. Pérez, N. García, and P. César. 2024. Delay threshold for social interaction in volumetric extended reality communication. ACM Transactions on Multimedia Computing, Communications and Applications, 20, 7, 1–22.

[3] E. d'Eon, B. Harrison, T. Myers, and P. A. Chou. 2017. 8i Voxelized Full Bodies - A Voxelized Point Cloud Dataset. ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG) input document WG11M40059/WG1M74006, 7, 8.
[4] M. De Fré, J. van der Hooft, C.-Y. Chang, K. De Schepper, P. R. Alface, D. De Vleeschauwer, T. Wauters, P. Steenkiste, and F. De Turck. 2025. Low-latency volumetric video conferencing in congested networks through l4s. In Proceedings of the 16th ACM Multimedia Systems Conference, 113–123.
[5] M. De Fré, J. van der Hooft, T. Wauters, and F. De Turck. 2024. Scalable MDC-based volumetric video delivery for real-time one-to-many WebRTC conferencing. In Proceedings of the 15th ACM Multimedia Systems Conference, 121–131.
[6] S. Dutton. 2013. Webrtc in the real world: stun, turn and signaling. Google, Nov, 1–22.
[7] Google. 2025. Draco. https://google.github.io/draco/. Accessed: 2025-09-11. (2025).
[8] Google. 2025. Real-Time Communication for Web. https://webrtc.org/. Accessed: 2025-08-30. (2025).
[9] D. Graziosi, O. Nakagami, S. Kuma, A. Zaghetto, T. Suzuki, and A. Tabatabai. 2020. An Overview of Ongoing Point Cloud Compression Standardization Activities: Video-Based (V-PCC) and GeometryBased (G-PCC). APSIPA Transactions on Signal and Information Processing, 9.
[10] B. Grozev, G. Politis, E. Ivov, T. Noel, and V. Singh. 2017. Experimental evaluation of simulcast for webrtc. IEEE Communications Standards Magazine, 1, 2, 52–59.
[11] S. N. B. Gunkel, R. Hindriks, K. M. E. Assal, H. M. Stokking, S. Dijkstra-Soudarissanane, F. Haar, and O. Niamut. 2021. VRComm: An End-to-End Web System for Real-Time Photorealistic Social VR Communication. In ACM Multimedia Systems Conference.
[12] imec. 2025. imec Virtual Wall. https://doc.ilabt.imec.be/ilabt/virtualwall/. Accessed: 2025-08-28. (2025).
[13] J. Jansen, T. Röggla, S. Rossi, I. Viola, and P. Cesar. 2024. OpenSourcing VR2Gather: A Collaborative Social VR System for Adaptive Multi-Party Real Time Communication. In ACM International Conference on Multimedia.
[14] J. Jansen, S. Subramanyam, R. Bouqueau, G. Cernigliaro, M. Cabré, F. Pérez, and P. Cesar. 2020. A Pipeline for Multiparty Volumetric Video Conferencing: Transmission of Point Clouds Over Low Latency DASH. In ACM Multimedia Systems Conference. Video Conferencing: Transmission of Point Clouds Over Low Latency DASH. In ACM Multimedia Systems Conference.
[15] J. Lawrence et al. 2021. Project Starline: A High-Fidelity Telepresence System. ACM Transactions on Graphics, 40, 6.
[16] L.-H. Lee, T. Braud, P. Y. Zhou, L. Wang, D. Xu, Z. Lin, A. Kumar, C. Bermejo, P. Hui, et al. 2024. All one needs to know about metaverse: a complete survey on technological singularity, virtual ecosystem, and research agenda. Foundations and Trends® in Human-Computer Interaction, 18, 2–3, 100–337.
[17] R. Mekuria, K. Blom, and P. Cesar. 2016. Design, Implementation, and Evaluation of a Point Cloud Codec for Tele-Immersive Video. IEEE Transactions on Circuits and Systems for Video Technology, 27, 4.
[18] MIT. 2025. Pion WebRTC. https://github.com/pion/webrtc. Accessed: 2025-09-11. (2025).
[19] J. Park, P. A. Chou, and J.-N. Hwang. 2019. Rate-utility optimized streaming of volumetric media for augmented reality. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 9, 1, 149–162.
[20] B. Sredojev, D. Samardzija, and D. Posarac. 2015. Webrtc technology overview and signaling solution design and implementation. In 2015 38th international convention on information and communication technology, electronics and microelectronics (MIPRO). IEEE, 1006–1009.
[21] S. Subramanyam, I. Viola, A. Hanjalic, and P. Cesar. 2020. User Centered Adaptive Streaming of Dynamic Point Clouds with Low Complexity Tiling. In ACM International Conference on Multimedia.
[22] S. Subramanyam, I. Viola, J. Jansen, E. Alexiou, A. Hanjalic, and P. Cesar. 2022. Evaluating the Impact of Tiled User-Adaptive RealTime Point Cloud Streaming on VR Remote Communication. In ACM International Conference on Multimedia.
[23] J. van der Hooft, H. Amirpour, M. Torres Vega, Y. Sanchez, R. Schatz, T. Schierl, and C. Timmerer. 2023. A Tutorial on Immersive Video Delivery: From Omnidirectional Video to Holography. IEEE Communications Surveys & Tutorials.
[24] J. van der Hooft, T. Wauters, F. De Turck, C. Timmerer, and H. Hellwagner. 2019. Towards 6dof http adaptive streaming through point cloud compression. In Proceedings of the 27th ACM International Conference on Multimedia, 2405–2413.
[25] I. Viola, J. Jansen, S. Subramanyam, I. Reimat, and P. Cesar. 2023. VR2Gather: A Collaborative Social VR System for Adaptive MultiParty Real-Time Communication. IEEE MultiMedia.
[26] I. Viola and P. Cesar. 2023. Volumetric video streaming: current approaches and implementations. Immersive Video Technologies, 425– 443.
[27] [SW], WebRTC SFU for Point Cloud Streaming 2025. url: https://github.com/idlab-discover/pc-webrtc-sfu.
[28] P. K. Yadav and W. T. Ooi. 2020. Tile rate allocation for 360-degree tiled adaptive video streaming. In Proceedings of the 28th ACM International Conference on Multimedia, 3724–3733.